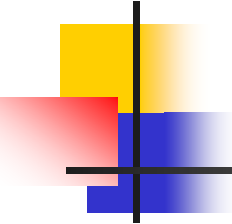




算法分析与设计

Analysis and Design of Algorithm

Lesson 15

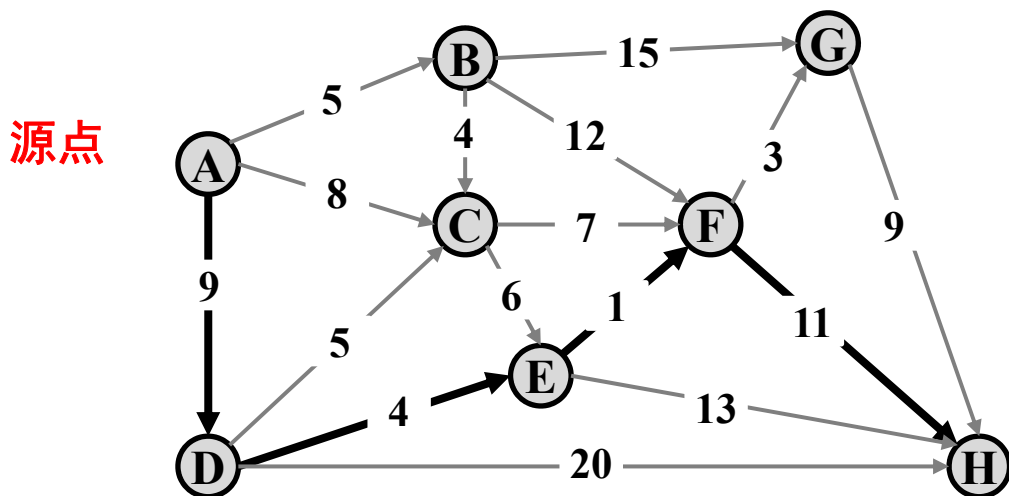


要点回顾

- 分支限界：一种与回溯法类似的算法
 - 将求解问题建模为搜索解空间树
 - 通常用代价(收益、限界、优先级)函数估算每个分支最优值
 - 优先选择当前看来最好的分支
 - 搜索策略一般采用宽度优先搜索
 - 搜索过程中剪枝（2个条件）
 - 不满足约束条件
 - 代价函数值不优于当前的界
- 分支限界实例
 - 一般背包问题、0-1背包问题
 - TSP问题、非对称TSP问题
 - 单源最短路径

单源最短路径（回顾）

- 给定带权有向图 $G=(V, E)$ ，其中每条边的权是非负实数。给定 V 中的一个顶点作为源点，求源点到所有其他各顶点的最短路长度。



计算距离(A,F)—分支限界法

■ 基本思想

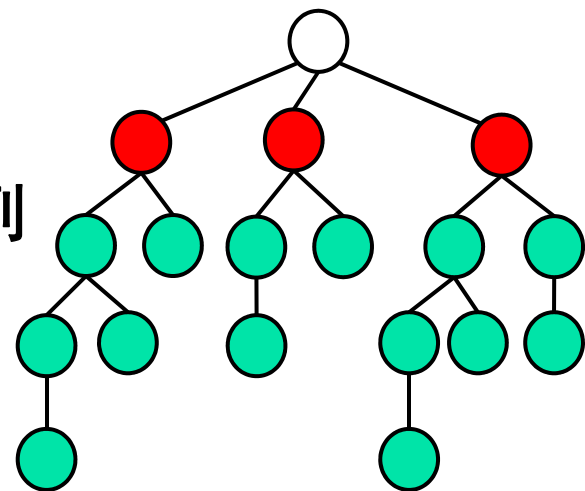
- 从源顶点s和仅含s的优先队列开始
- 节点队列中**选择**一节点，并**扩展**
- 若节点不被剪枝，将节点**插入**节点队列
- 反复2~3步，直到队列为**空时止**

■ 剪枝函数

- 节点*i*代表顶点*v*的一个距离 $dist(v)$
- 若 $dist(v) \geq$ 源点到终点最短距离，则剪枝
- 若 $dist(v) \geq dist_best(v)$ ，则剪枝

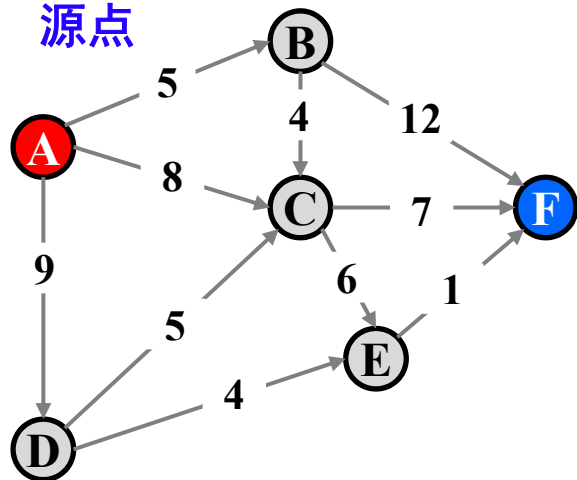
■ 节点选择方法

1. 先入先出（FIFO队列）
2. 当前路径长度最短（优先级队列）



计算距离(A,F)—FIFO队列

源点

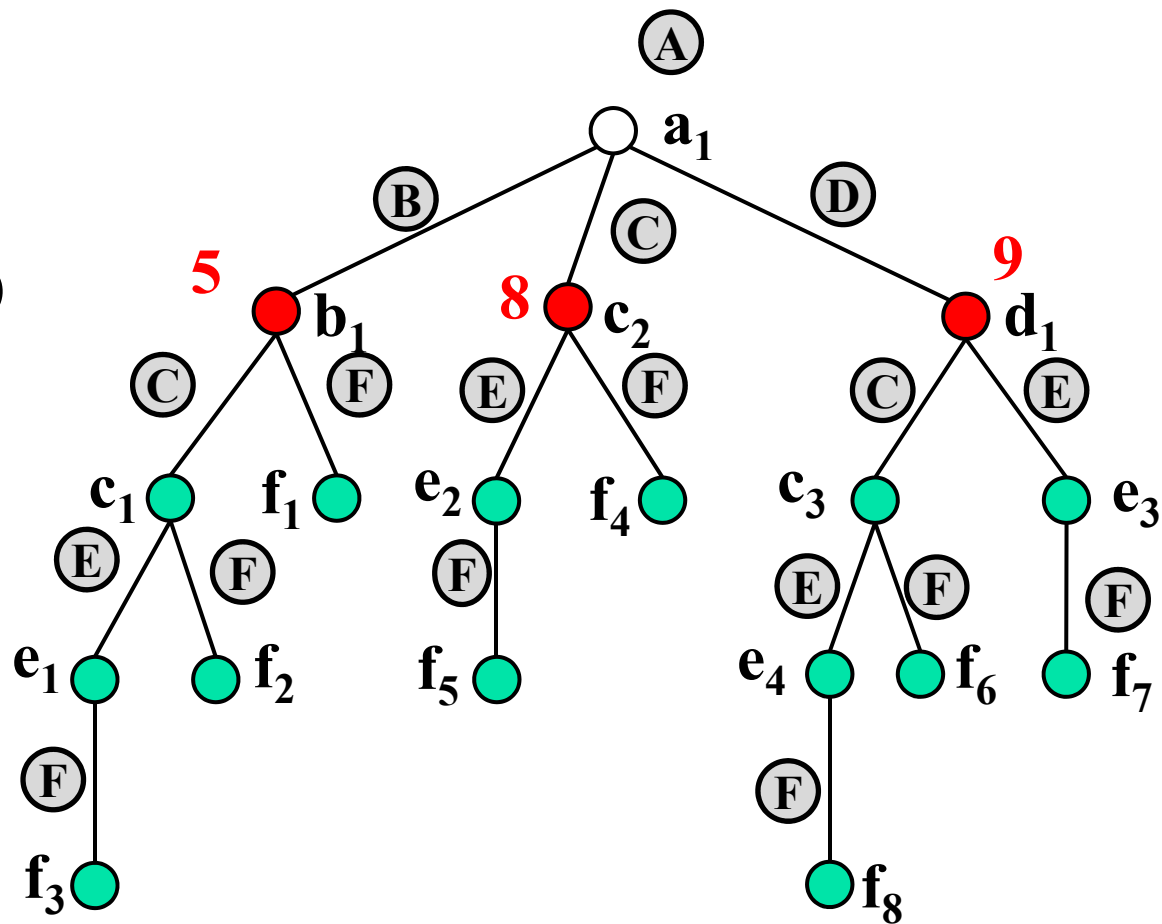


FIFO队列

	b₁	c ₂	d ₁		
dist	5	8	9		

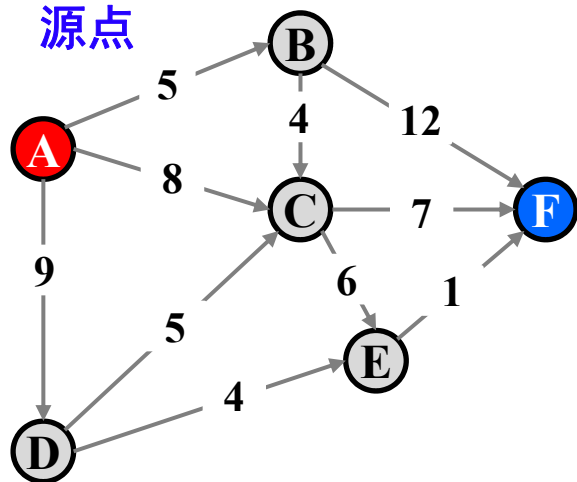
当前最短距离

	B	C	D	E	F
dist_best	5	8	9		



计算距离(A,F)—FIFO队列

源点

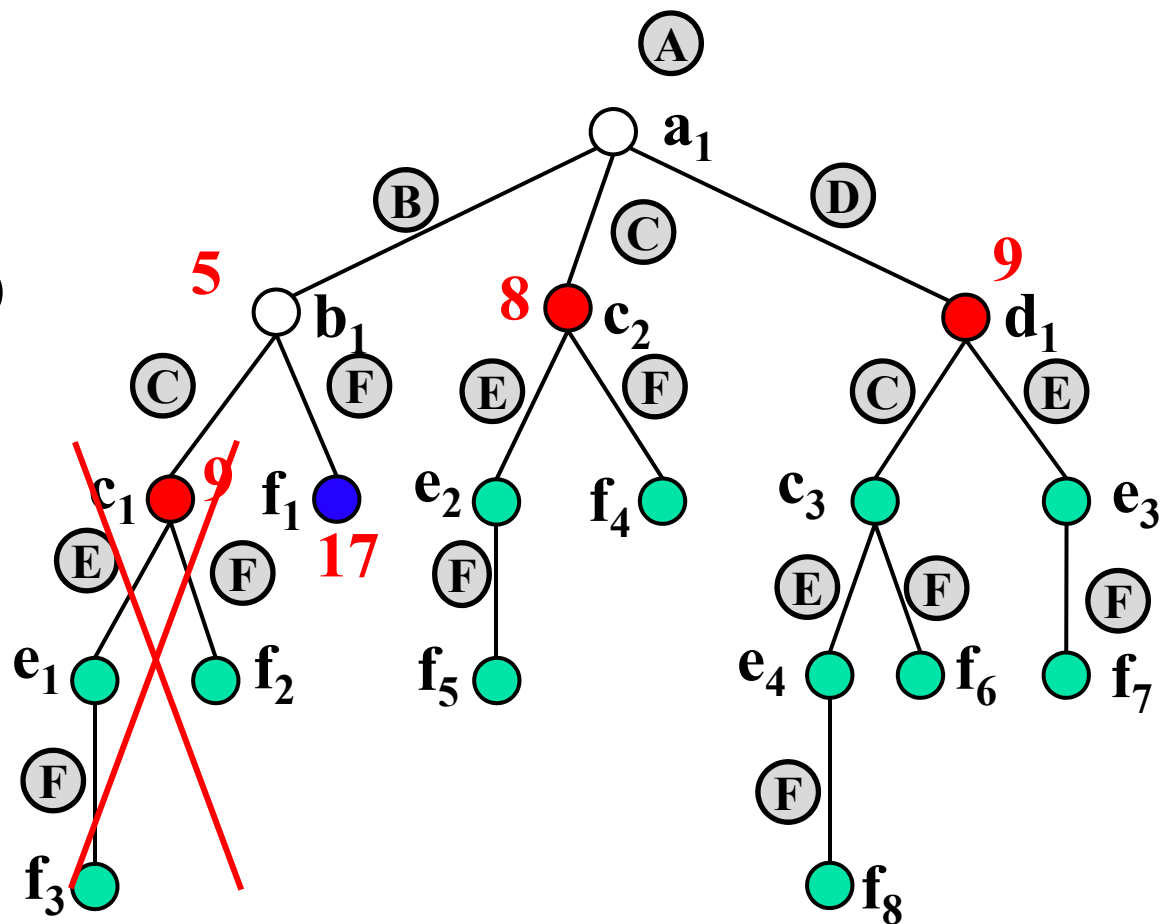


FIFO队列

	c_2	d_1			
dist	8	9			

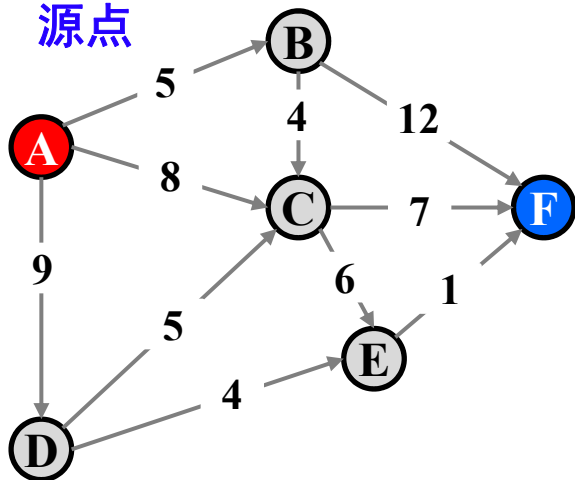
当前最短距离

	B	C	D	E	F
dist_best	5	8	9		17



计算距离(A,F)—FIFO队列

源点

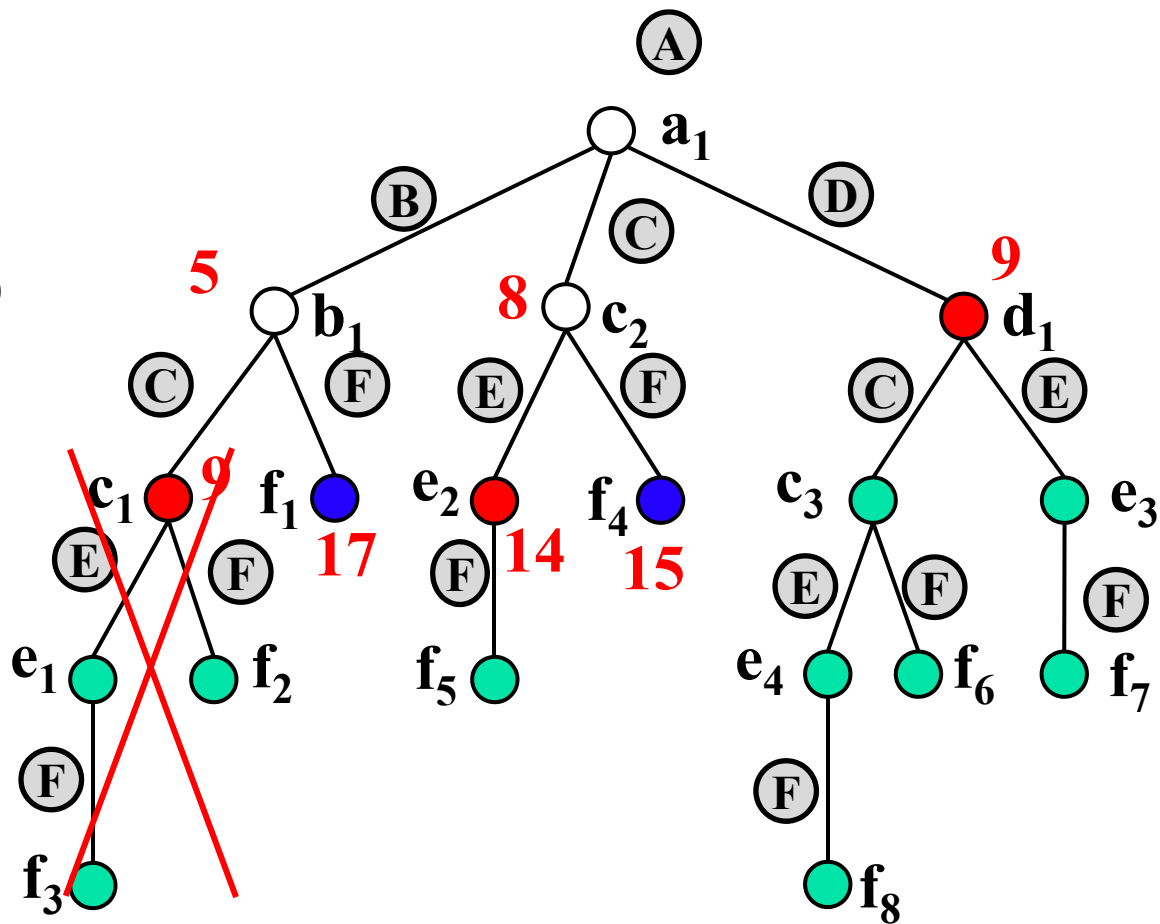


FIFO队列

	d₁	e ₂			
dist	9	14			

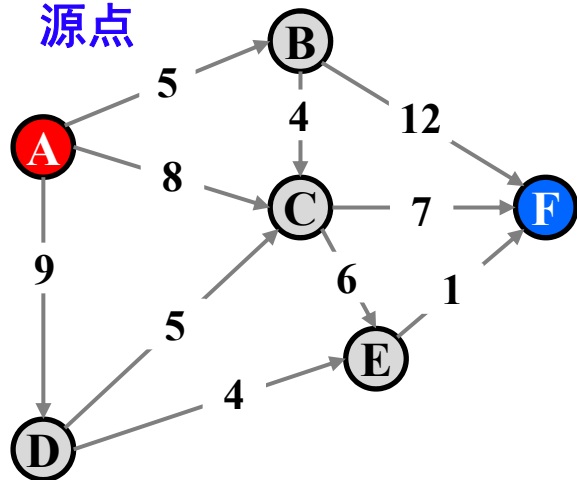
当前最短距离

	B	C	D	E	F
dist_best	5	8	9	14	15



计算距离(A,F)—FIFO队列

源点

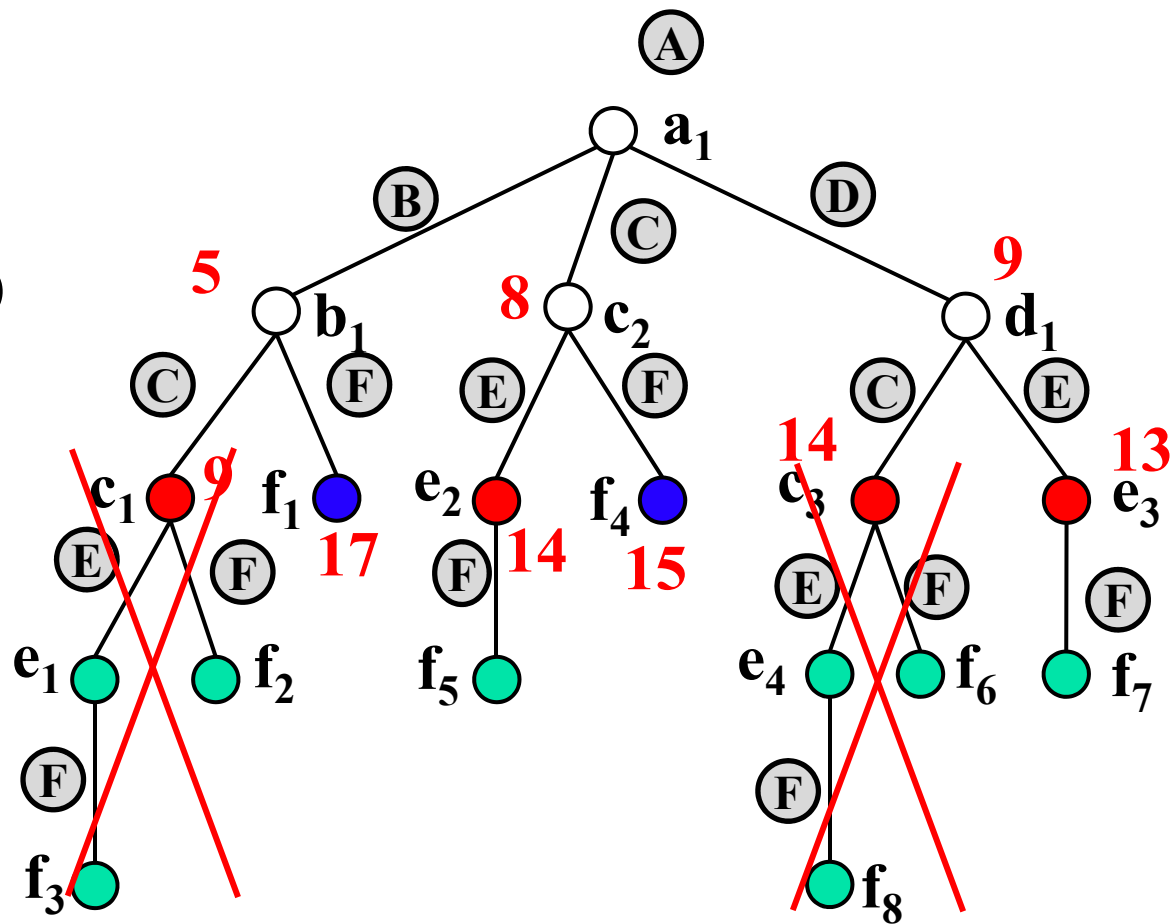


FIFO队列

	e_2	e_3			
dist	14	13			

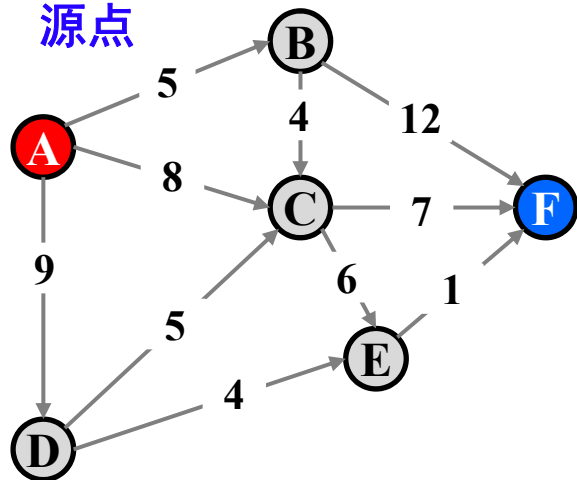
当前最短距离

	B	C	D	E	F
dist_best	5	8	9	13	15



计算距离(A,F)—FIFO队列

源点

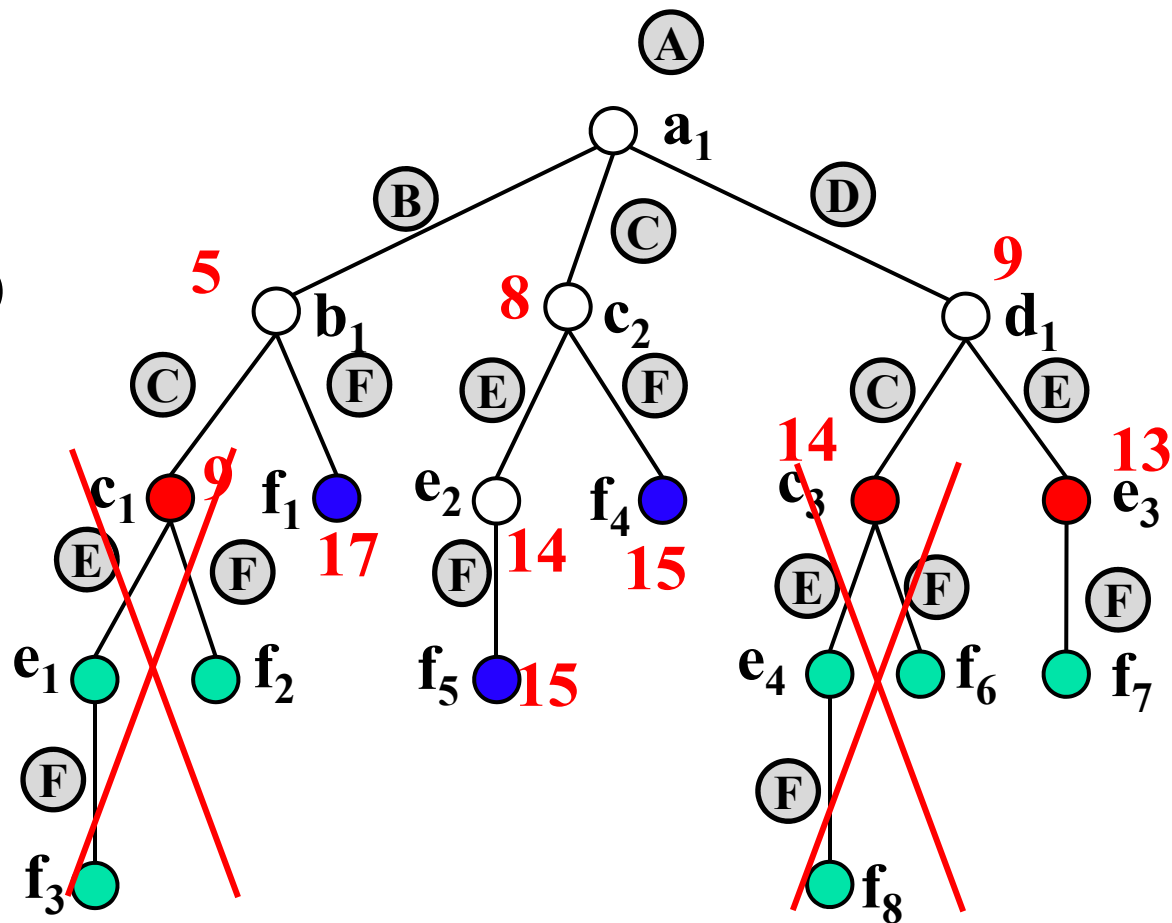


FIFO队列

	e₃				
dist	13				

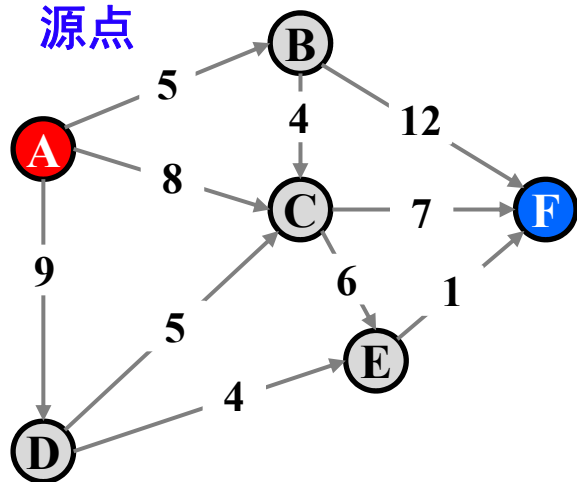
当前最短距离

	B	C	D	E	F
dist_best	5	8	9	13	15



计算距离(A,F)—FIFO队列

源点

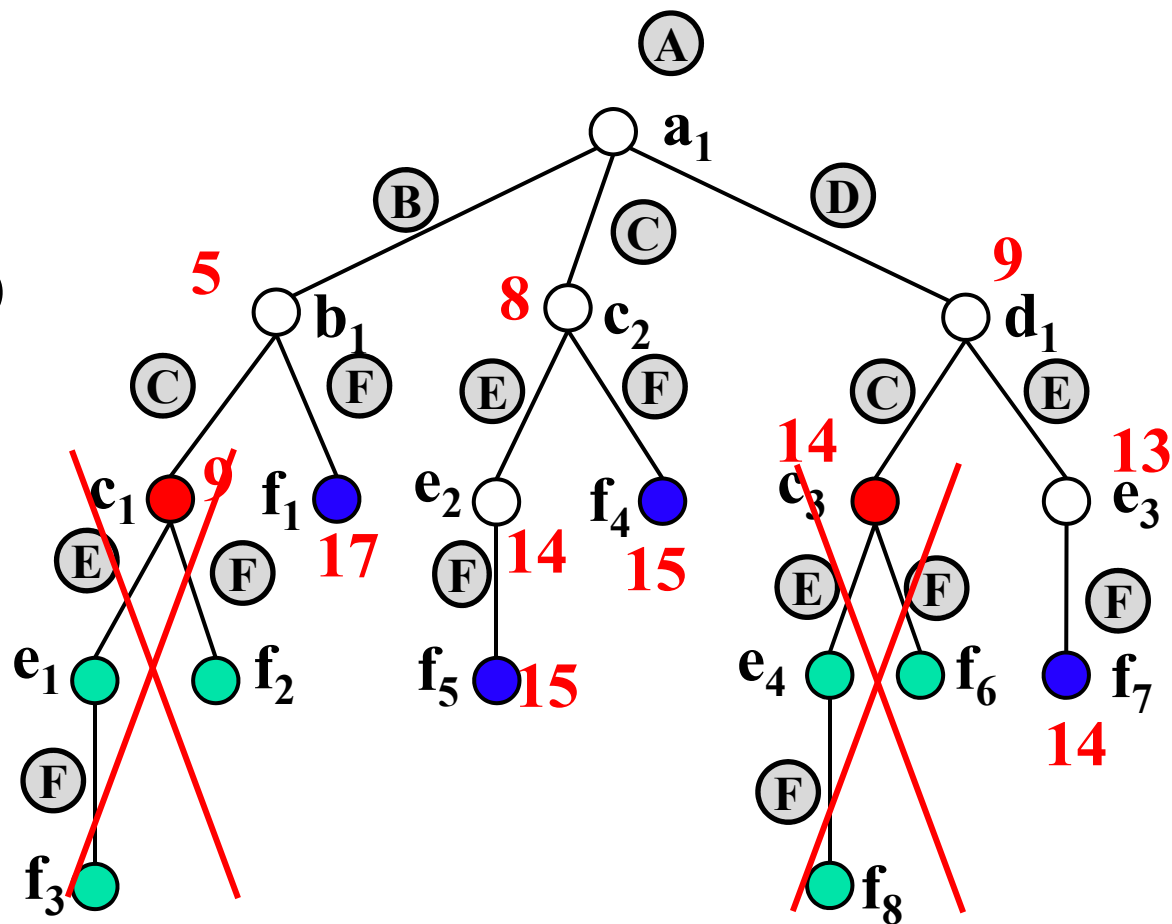


FIFO队列

dist					

当前最短距离

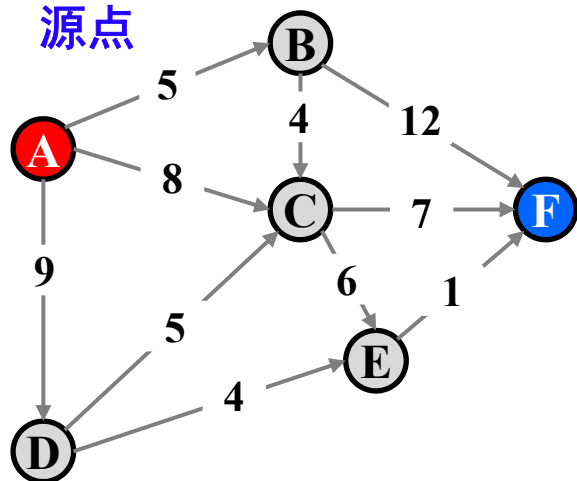
	B	C	D	E	F
dist_best	5	8	9	13	14



当FIFO队列空时，
停止分支限界法

计算距离(A,F)—优先级队列

源点

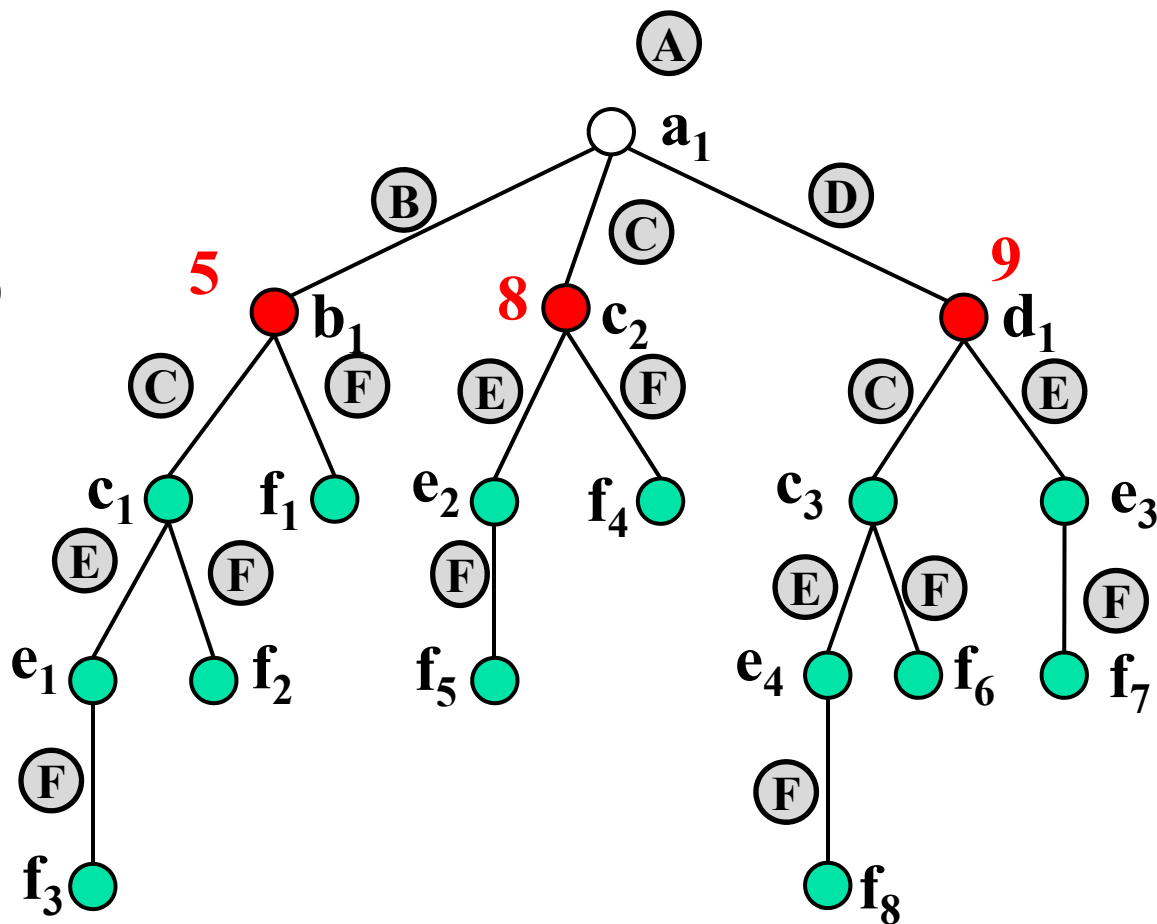


优先级队列

	b_1	c_2	d_1		
dist	5	8	9		

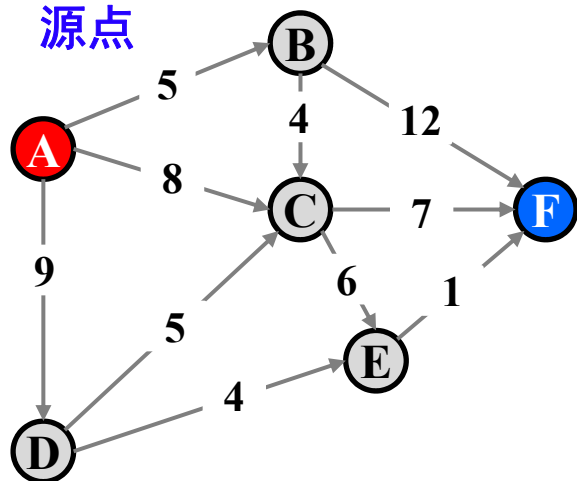
当前最短距离

	B	C	D	E	F
dist_best	5	8	9		



计算距离(A,F)—优先级队列

源点

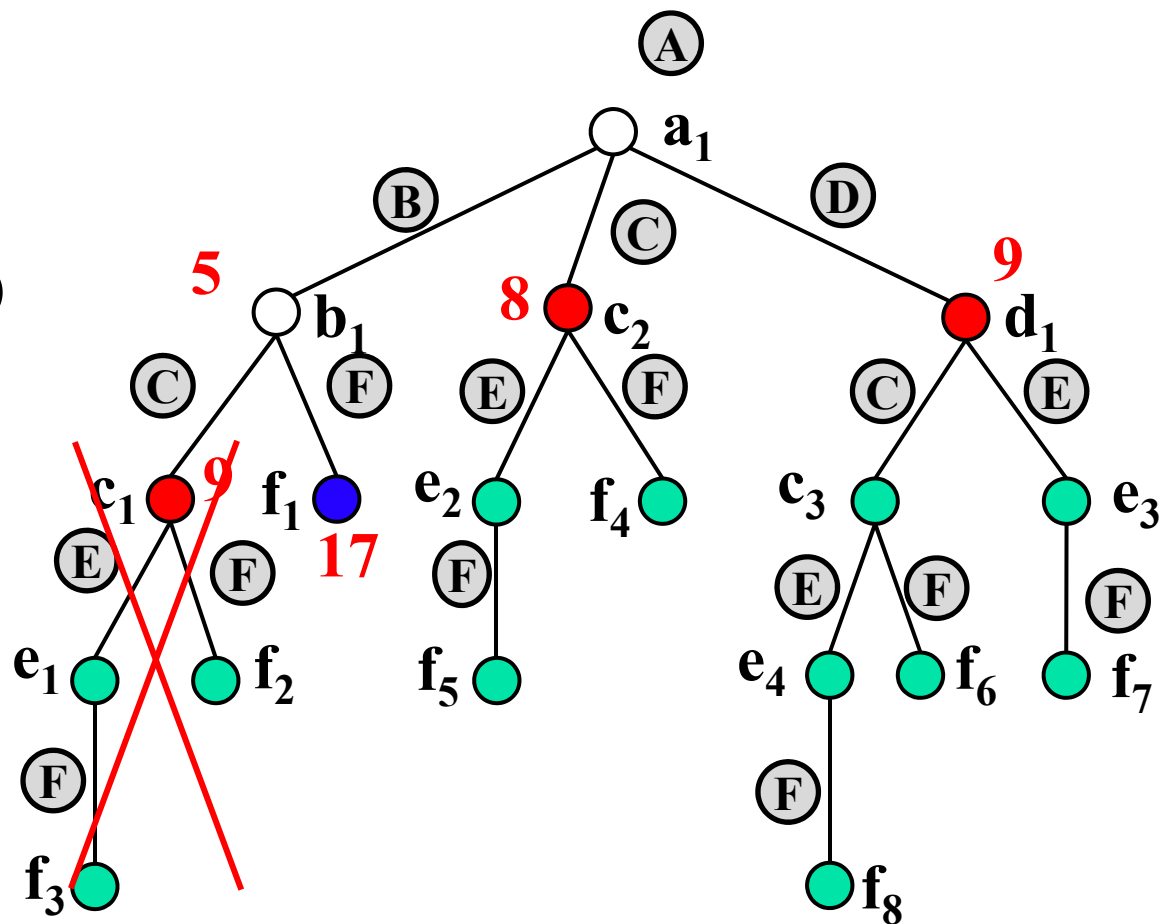


优先级队列

	c_2	d_1			
dist	8	9			

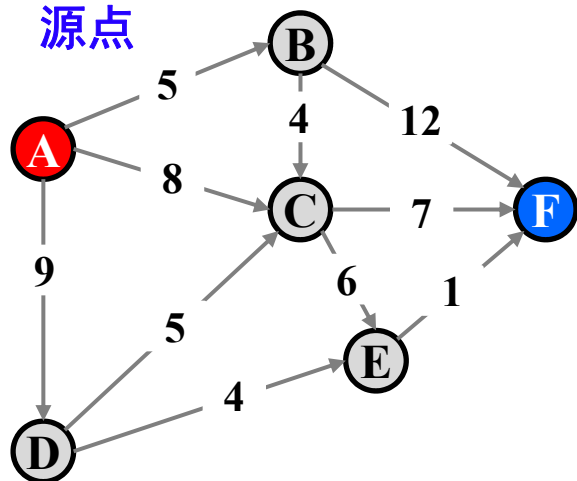
当前最短距离

	B	C	D	E	F
dist_best	5	8	9		17



计算距离(A,F)—优先级队列

源点

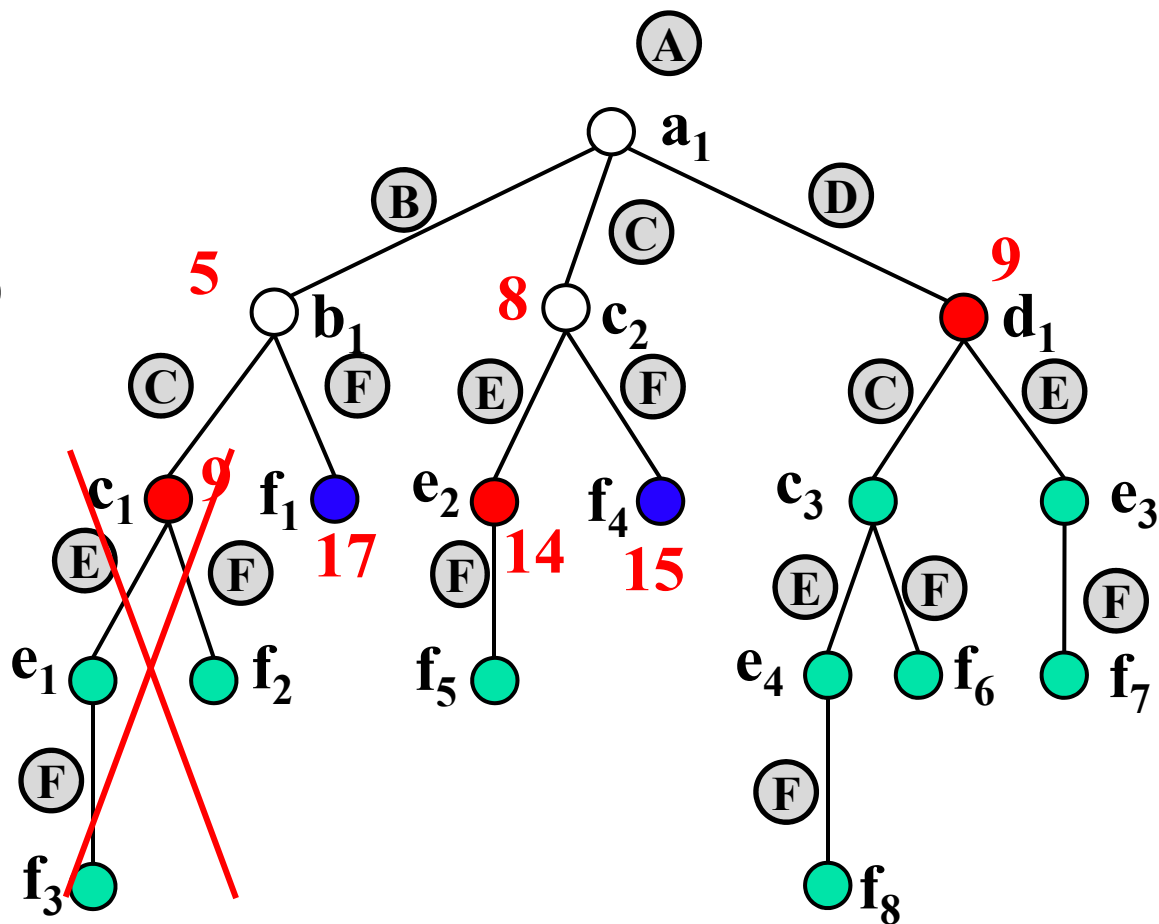


优先级队列

	d₁	e ₂			
dist	9	14			

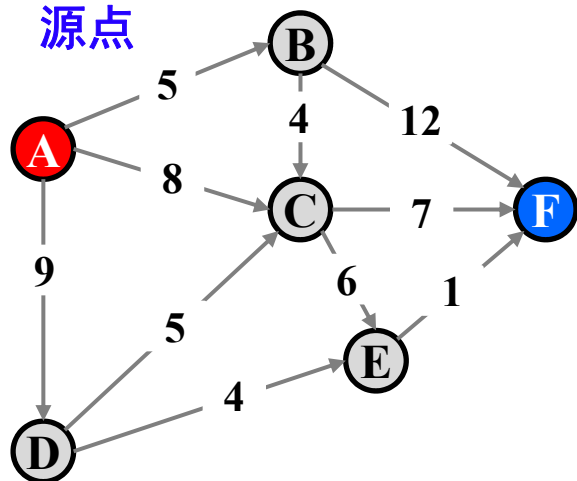
当前最短距离

	B	C	D	E	F
dist_best	5	8	9	14	15



计算距离(A,F)—优先级队列

源点

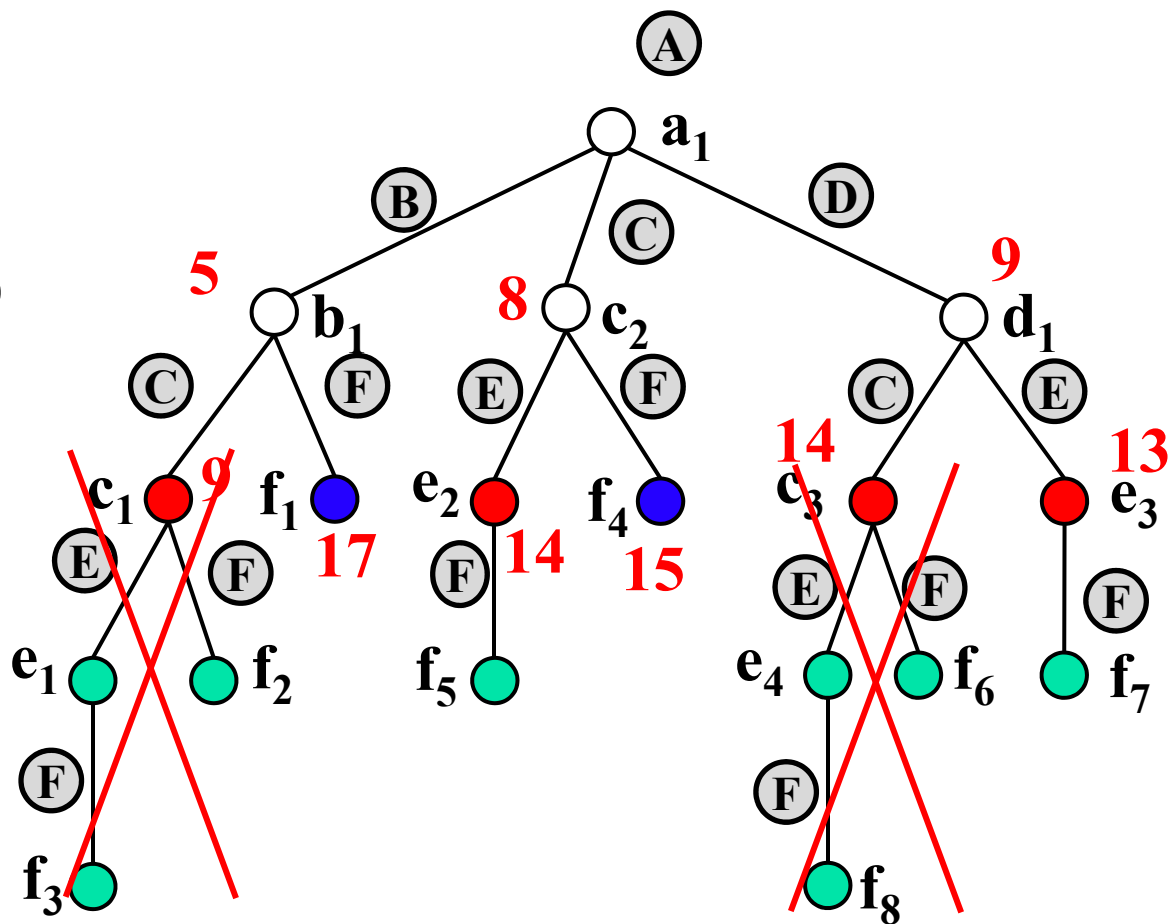


优先级队列

	e_3	e_2			
dist	13	14			

当前最短距离

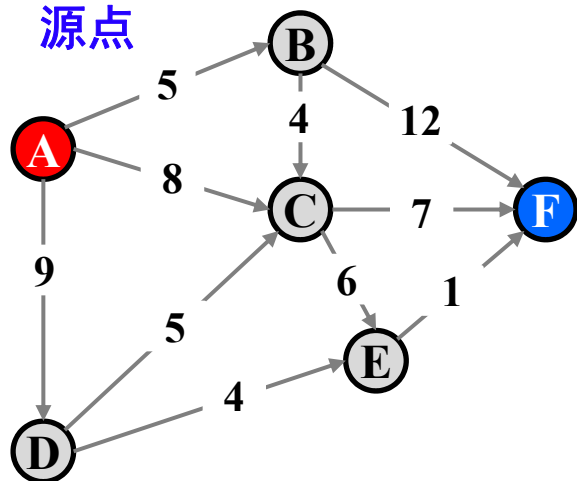
	B	C	D	E	F
dist_best	5	8	9	13	15



与FIFO队列区别在于先扩展 e_3

计算距离(A,F)—优先级队列

源点

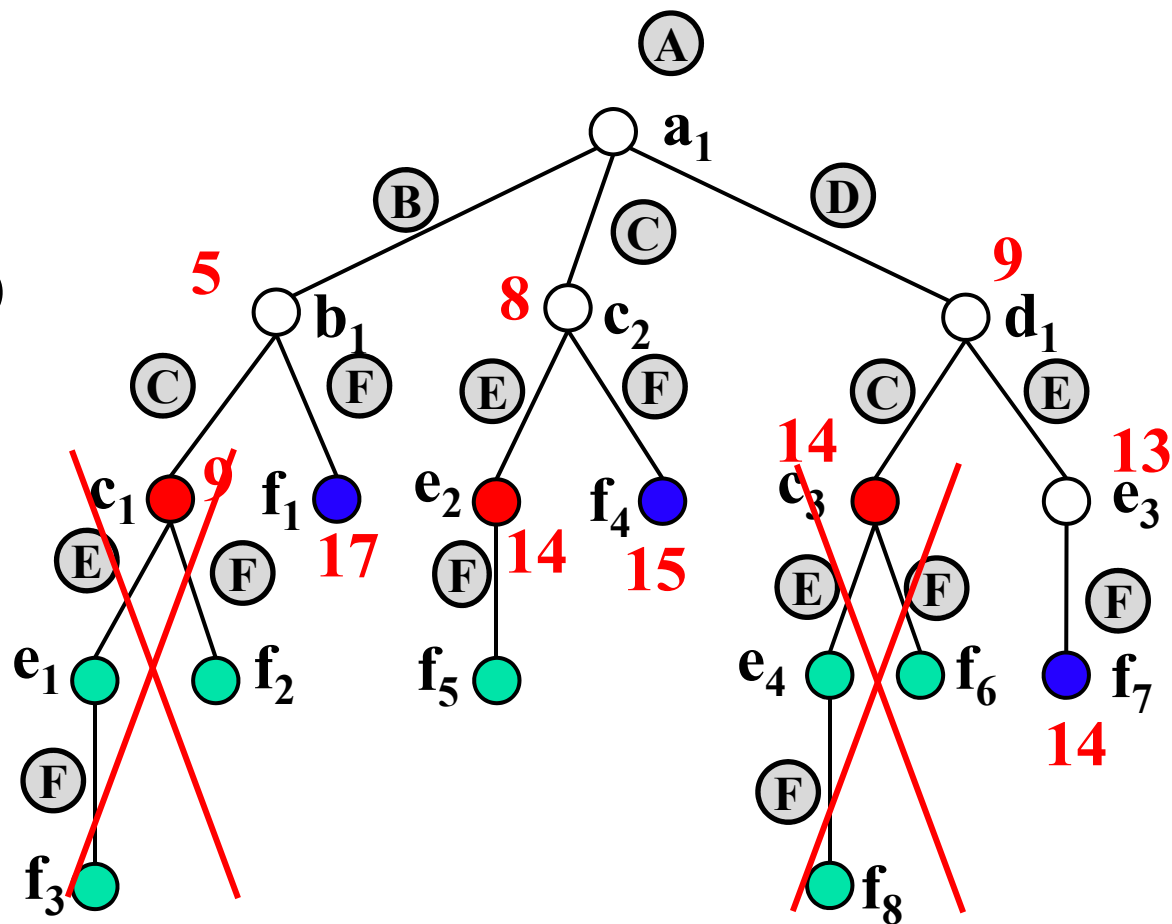


优先级队列

	e_2				
dist	14				

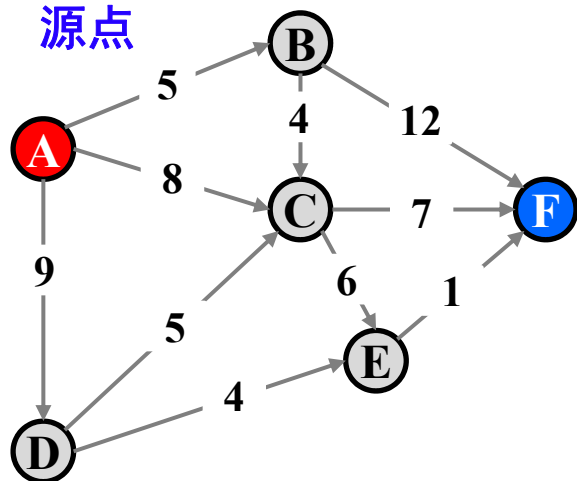
当前最短距离

	B	C	D	E	F
dist_best	5	8	9	13	14



计算距离(A,F)—优先级队列

源点

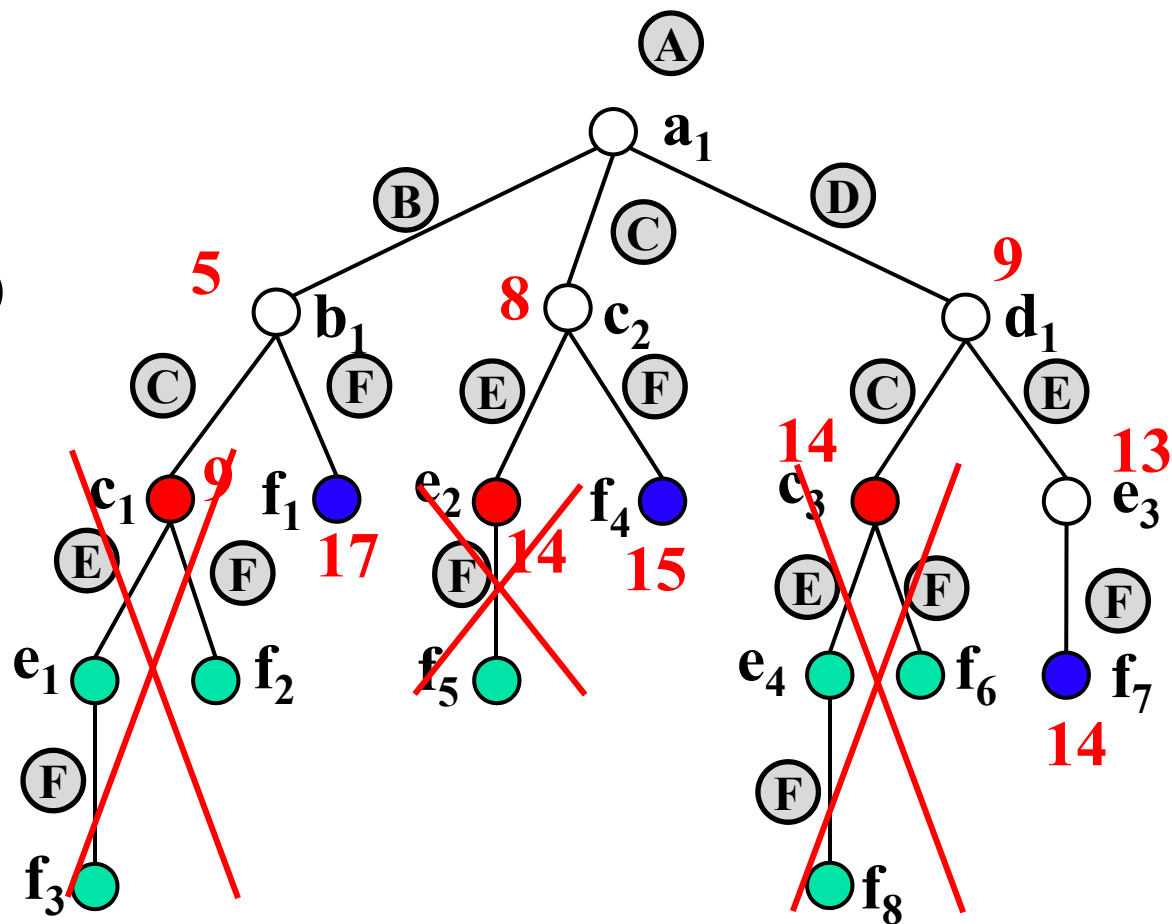


优先级队列

dist					

当前最短距离

	B	C	D	E	F
dist_best	5	8	9	13	14



当优先级队列空时，
停止分支限界法

最短路径问题—算法的比较

遍历解空间树的方法，如何选择下一个节点

■ Dijkstra算法

- 当前步的最优，复杂度为 $O(nm)$
- 不适合设计分布式算法

■ 回溯法

- 当前路径的下一跳(深度优先搜索)

■ 分支限界算法

- 当前图中跳数的最优(FIFO队列，广度优先搜索)
- 当前距离的最优(优先级队列，优先级搜索)

→相较于
Dijkstra算法，
分支限界算法
更适合设计分
布式的单源最
短路径算法

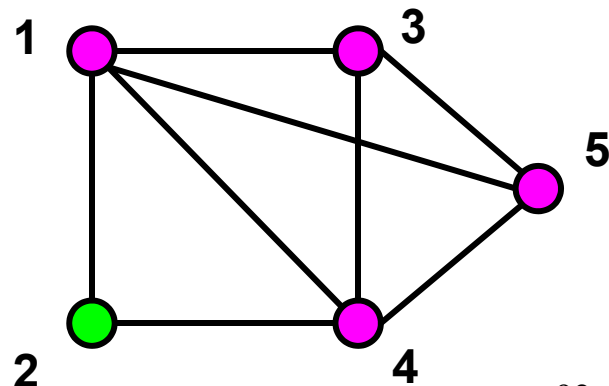
最大团问题

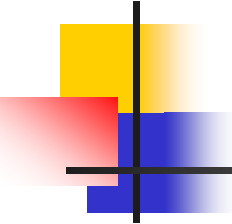
- 无向图 $G=\langle V,E \rangle$ 相关的几个概念
 - G 的**子图**: $G'=\langle V',E' \rangle$, $V'\subseteq V, E'\subseteq E$
 - G 的**补图**: $\overline{G}=\langle V,E' \rangle$, E' 是 E 关于完全图边集的补集
 - G 中的**团**: G 的完全子图
 - G 的**最大团**: 顶点数最多的团

- 实例:

团: $\{1,2,4\}, \{1,3,4\}, \{3,4,5\},$
 $\{1,3,5\}, \{1,4,5\}, \{1, 3, 4, 5\}$

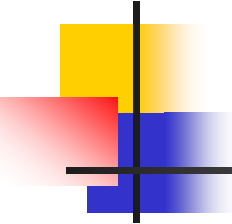
最大团: $\{1, 3, 4, 5\}$





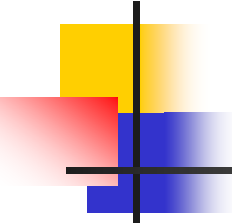
最大团问题

- **问题：** 给定无向图 $G=\langle V,E \rangle$ ，其中顶点集 $V=\{1,2, \dots, n\}$ ，边集为 E ，求 G 的最大团。
- **解：** $\langle x_1, x_2, \dots, x_n \rangle$ 为 0-1 向量， $x_k=1$ 当且仅当顶点 k 属于最大团
- **穷举法：** 对每个顶点子集，检查是否构成团，即其中每对顶点之间是否都有边。有 2^n 个子集，至少需要指数时间。



分支限界算法设计

- 搜索数为**子集树**
- 结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的含义：
 - 已检索顶点 $1, 2, \dots, k$ ，其中 $x_i=1$ 表示顶点 i 在当前的团内
- **约束条件**：该顶点与当前团内每个顶点都有边相连
- **界**：当前已检索到的极大团的顶点数



剪枝函数

- **剪枝函数：** 目前的团可能扩张为极大团的顶点数上界

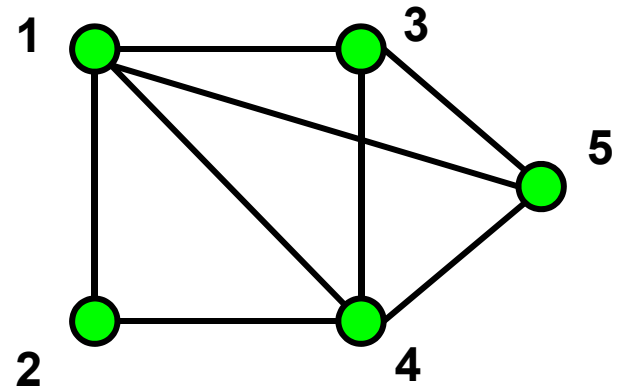
$$F = C_k + n - k$$

其中 C_k 为当前团的顶点数（初始为0）， k 为结点层数

最坏情况下时间： $O(n2^n)$

实例

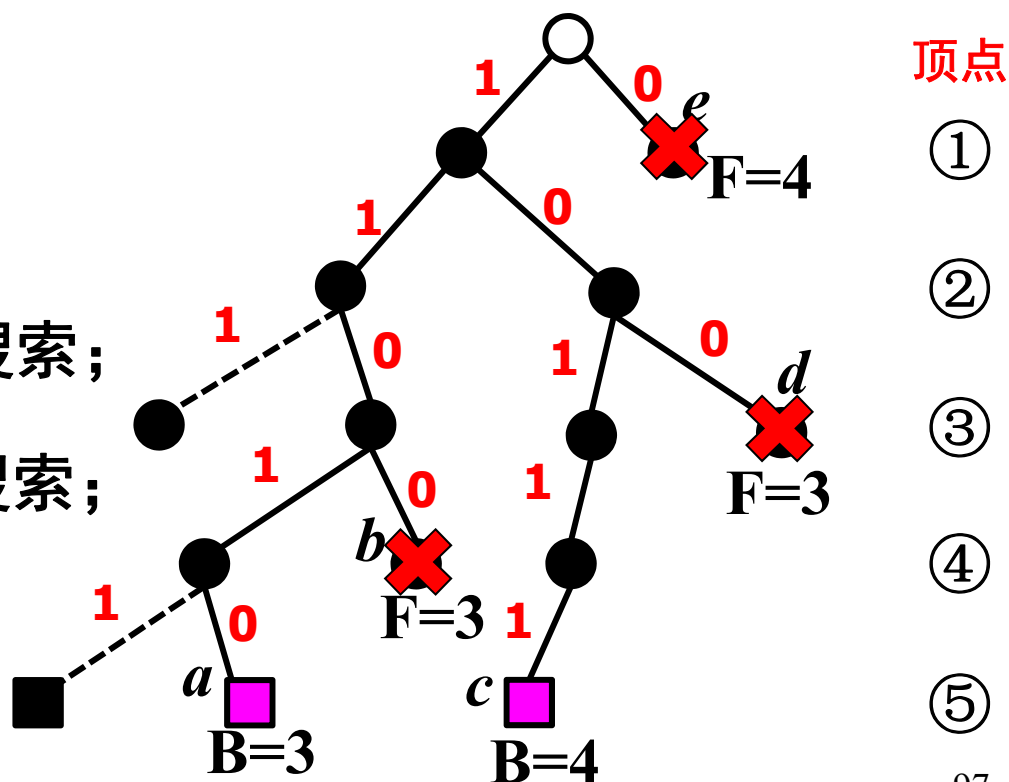
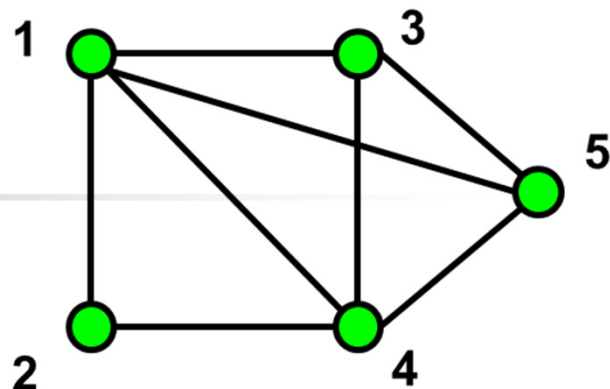
- 顶点编号顺序为1, 2, 3, 4, 5
- 对应 x_1, x_2, x_3, x_4, x_5
- $x_i=1$ 当且仅当 i 在团内
- 分支规定左子树为1, 右子树为0
- B为界, F为剪枝函数值



搜索树

- *a*: 极大团{1, 2, 4}, 顶点数为3, 界为 $B=3$
- *b*: 剪枝函数值 $F=3$, 回溯;
- *c*: 极大团{1, 3, 4, 5}, 顶点数为4, 界为 $B=4$
- *d*: 剪枝函数值 $F=3$, 不必搜索;
- *e*: 剪枝函数值 $F=4$, 不必搜索;

输出最大团{1, 3, 4, 5}
顶点数为4



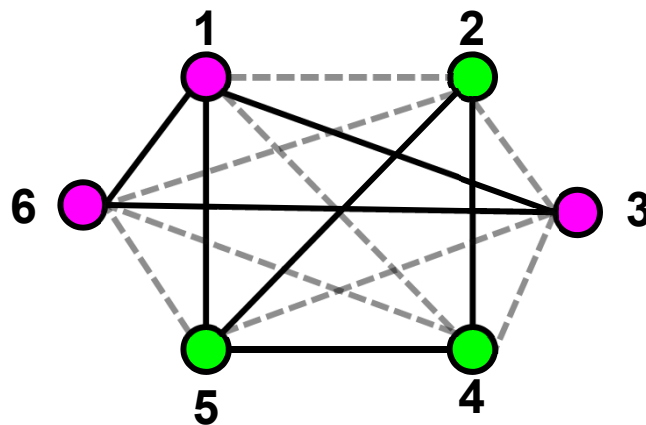
独立集与团

- G的**点独立集**：G的顶点子集A，且
$$\forall u, v \in A, \{u, v\} \notin E$$
- **最大点独立集**：顶点最多的点独立集
- **命题**：U是G的最大团当且仅当U是 $\bar{G}$ 的最大点独立集

G的最大团：

$U = \{1, 3, 6\}$

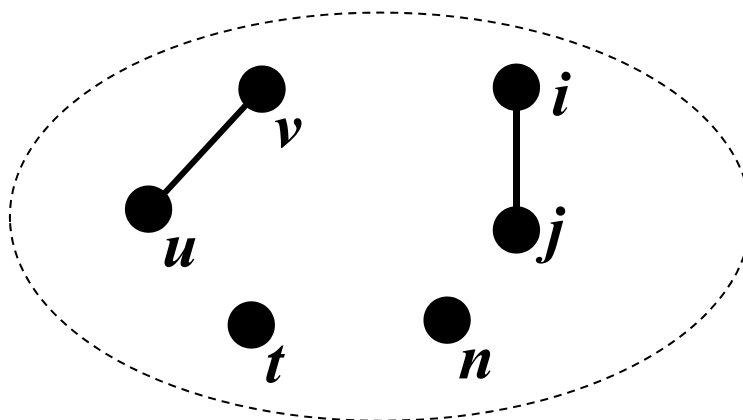
补图 $\bar{G}$ 的最大点独立集



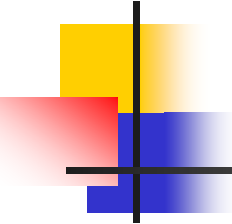
应用

- 编码、故障诊断、计算机视觉、聚类分析、经济学、移动通信、VLSI电路设计。。。
- 例子：噪音使信道传输字符发生混淆
- **混淆图** $G=\langle V,E \rangle$, V 为有穷字符集, $\{u,v\} \in E \Leftrightarrow u$ 和 v 易混淆

$G=\langle V,E \rangle$

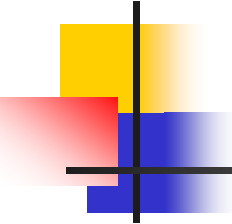


为减少噪音干扰，设计编码应该找到混淆图中的最大点独立集



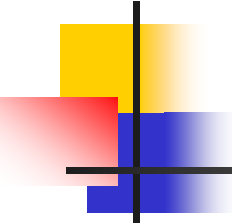
第六章小结

- 分支限界：一种与回溯法类似的算法
 - 将问题建模为解空间树
 - 通常用限界/剪枝函数估算每个分支的最优值
 - 优先选择当前看来最好的分支
 - 搜索策略一般采用宽度优先搜索
 - 搜索过程中剪枝
- 分支限界的剪枝函数
 - 不满足约束条件
 - 剪枝函数值不优于当前的界



分支限界法与回溯法的区别

- **搜索方式**不同
 - 回溯法：深度优先
 - 分支限界法：FIFO队列式(宽度优先)、优先级队列式
- **搜索目标**不同
 - 回溯法：找所有解、可行解、最优解
 - 分支限界法：找最优解
- **搜索用到的函数**不同
 - 回溯法：剪枝函数
 - 分支限界：剪枝函数+代价、限界、优先级函数
- **实例**：背包问题、TSP问题、非对称TSP问题、单源最短路径问题、最大团问题



课程内容

NP完全性理论与近似算法

算法高级理论

随机化算法

线性规划与网络流

高级算法

递归
分治

动态
规划

贪心
算法

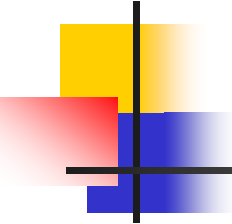
回溯与
分支限界

基础算法

算法分析与问题的计算复杂性

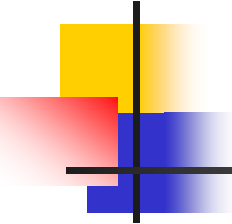
算法基础理论

第七章 随机化算法



学习要点

- 理解产生伪随机数的算法
- 掌握数值随机算法的设计思想
- 掌握Sherwood算法的设计思想
- 掌握Las Vegas算法的设计思想
- 掌握Monte Carlo算法的设计思想



随机算法的基本概念

■ 例子：

- 判断函数 $f(x_1, x_2, \dots, x_n)$ 在区域 D 中是否恒大于0， f 很复杂，不能数学化简，如何判断就很麻烦
 - 若随机产生一个 n 维坐标 $(r_1, r_2, \dots, r_n) \in D$ ，代入得 $f(r_1, r_2, \dots, r_n) < 0$ ，则可判定区域 D 内 f 不恒大于0
 - 若对很多个随机产生的坐标进行测试，结果次次均大于0，则可说 $f > 0$ 的概率是非常大
- 有不少问题，目前只有效率很差的确定性求解算法，但用随机算法去求解，可以很快地获得相当可信的结果

伪随机数的概念

■ 伪随机数的生成方法

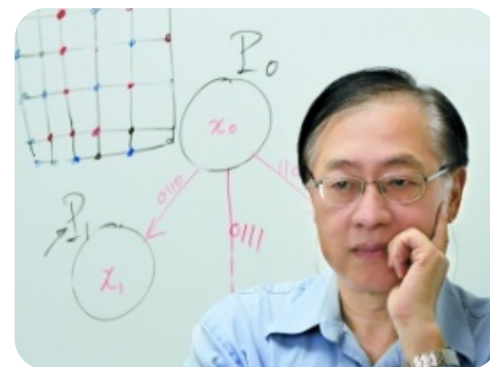
- C++: `srand(seed); int x=rand();`
- Java: `Random`类
- Python: `import random`

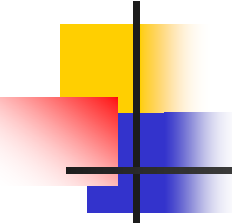
■ 真正的随机数

- 不可预测的，不可见的

■ 计算机中的伪随机数

- 按照一定算法模拟产生，是“确定的，可见的”
- 姚期智（2000年图灵奖，伪随机数生成的贡献）





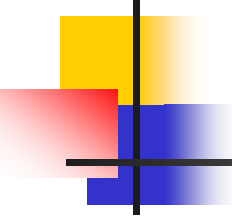
随机算法的基本概念

■ 随机算法

- 随机算法是一种使用概率和统计方法在其执行过程中对于下一计算步骤作出**随机选择**的算法

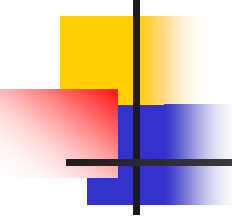
■ 随机算法的优越性

- 对于有些问题：算法**简单**
- 对于有些问题：时间**复杂性低**
- 对于有些问题：同时兼有简单和时间复杂性低



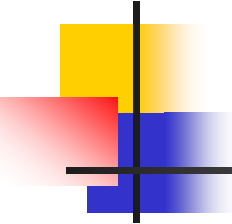
随机算法四兄弟

1. 随机数值算法
2. Sherwood算法
3. Las Vegas算法
4. Monte Carlo算法



随机算法四兄弟

1. 随机数值算法
2. Sherwood算法
3. Las Vegas算法
4. Monte Carlo算法



1. 随机数值算法

- 主要用于数值问题求解
- 算法的输出往往是近似解
- 近似解的精确度与算法执行时间成正比
- 应用范例
 - 计算 π 值
 - 计算定积分
 - 解线性方程组



山巔一寺一壺酒，尔乐苦煞吾，
把酒吃，酒杀尔杀不死，乐尔乐

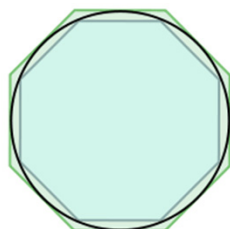
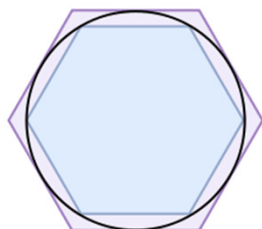
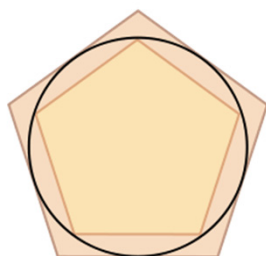
3.14159,26535,897,932384,626

π 的故事

祖衝之（四二九—五〇〇），字文遠，範陽適縣人（今河北涑水縣北）人。南北朝時代著名科學家。他推算出圓周率的數值在3.1415926和3.1415927之間，比歐洲人早一千多年。



祖衝之

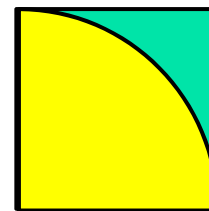
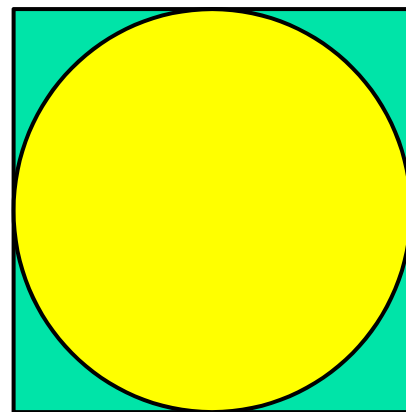


割圆术

用随机数值算法计算 π 值

- 设有一半径为 r 的圆及其外切四边形。
- 向该正方形随机地投掷 n 个点，落入圆内的点数为 k
- 当 n 足够大时， k 与 n 之比接近逼近 $\frac{\pi}{4}$ ，即 $\pi \approx \frac{4k}{n}$

```
double Darts(int n)
{ // 用随机投点法计算 $\pi$ 值
  static RandomNumber dart;
  int k=0;
  for (int i=1; i<=n; i++) {
    double x=dart.fRandom();
    double y=dart.fRandom();
    if ((x*x+y*y)<=1) k++;
  }
  return 4*k/double(n);
}
```

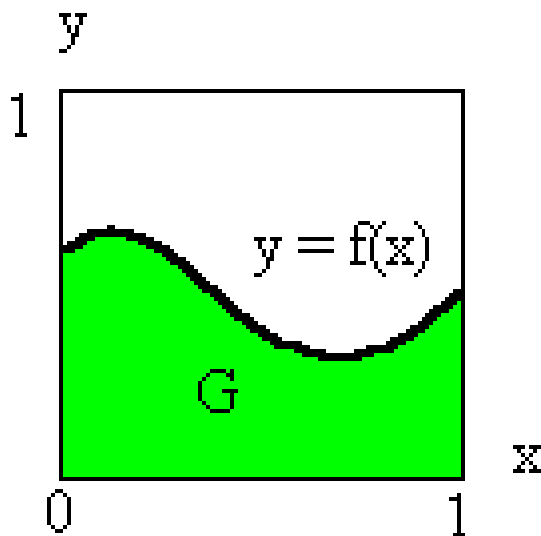


只用计算1/4
的部分

计算定积分

- 设 $f(x)$ 是 $[0, 1]$ 上的连续函数，且 $0 \leq f(x) \leq 1$ 。

- 需要计算的积分为 $I = \int_0^1 f(x) dx$,积分 I 等于图中的面积 G 。

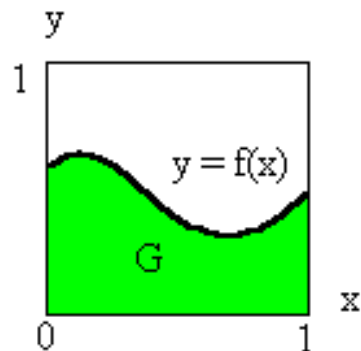


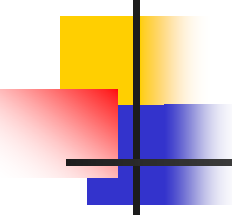
计算定积分

- 在图所示单位正方形内均匀地作投点试验，则随机点落在曲线下方的概率为

$$P_r \{y \leq f(x)\} = \int_0^1 \int_0^{f(x)} dy dx = \int_0^1 f(x) dx$$

- 假设向单位正方形内随机地投入 n 个点 (x_i, y_i) 。如果有 m 个点落入 G 内，则随机点落入 G 内的概率 $I \approx \frac{m}{n}$



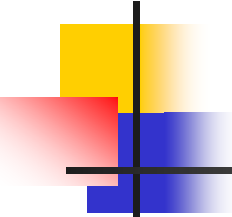


随机算法四兄弟

1. 随机数值算法
2. **Sherwood**算法
3. Las Vegas算法
4. Monte Carlo算法

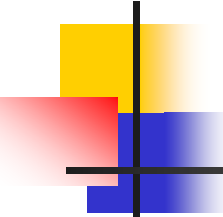
2. Sherwood算法

- 设 A 是一个确定性算法，当它的输入实例为 x 时所需的计算时间记为 $t_A(x)$ 。设 X_n 是算法 A 的输入规模为 n 的实例的全体，则当问题的输入规模为 n 时，算法 A 所需的平均时间为
$$\bar{t}_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$$
- 这显然不能排除存在 $x \in X_n$ 使得 $t_A(x) \gg \bar{t}_A(n)$ 的可能性。希望获得一个概率算法 B ，使得对问题的输入规模为 n 的每一个实例均有
$$t_B(x) = \bar{t}_A(n) + s(n)$$
- 这就是**Sherwood算法**设计的基本思想。当 $s(n)$ 与 $t_A(n)$ 相比可忽略时，舍伍德算法可获得很好的平均性能。



2. Sherwood算法

- 分析一个算法在平均情况下的计算复杂性，常假定算法的输入服从某一特定的概率分布。
- 例如，数据均匀分布时，快速排序的平均时间为 $O(n\log n)$ ，而输入基本有序时是 $O(n^2)$ 。
- 可用**Sherwood算法**来消除算法的时间复杂性与输入实例间的某种联系。
- 如果一个确定性算法无法直接改造成Sherwood算法，可用随机预处理技术，不改变原有的确定性算法，仅对其输入实例随机排列（洗牌）。



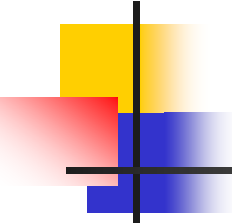
2. Sherwood算法

■ 性质：

- 一定能够求得一个正确解
- 算法最坏与平均复杂性差别大时, 加随机性
- 消除最坏行为与特定实例的联系

■ 应用范例：

- 线性时间选择算法
- 快速排序算法
- 搜索有序表
- 跳跃表

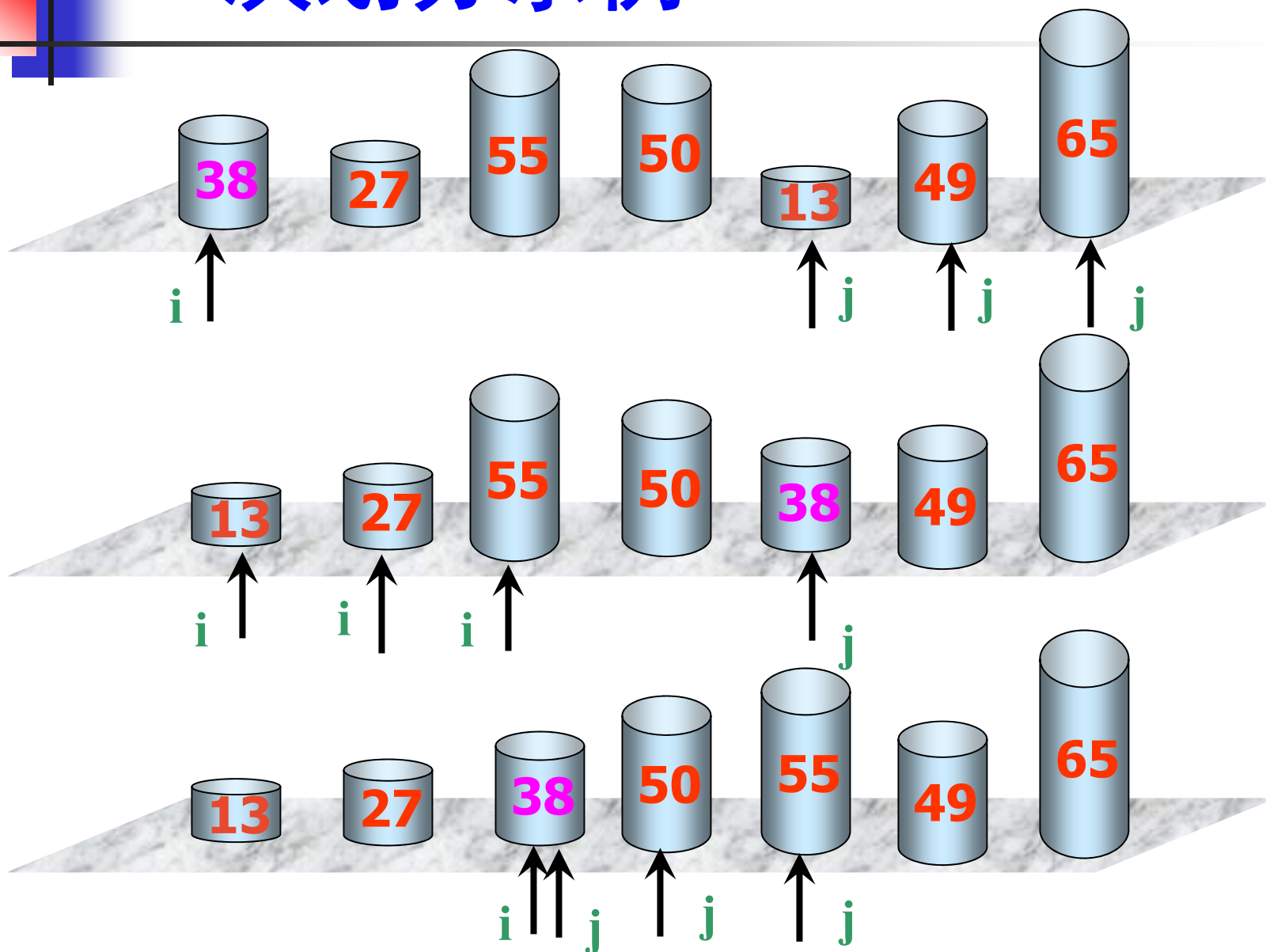


快速排序

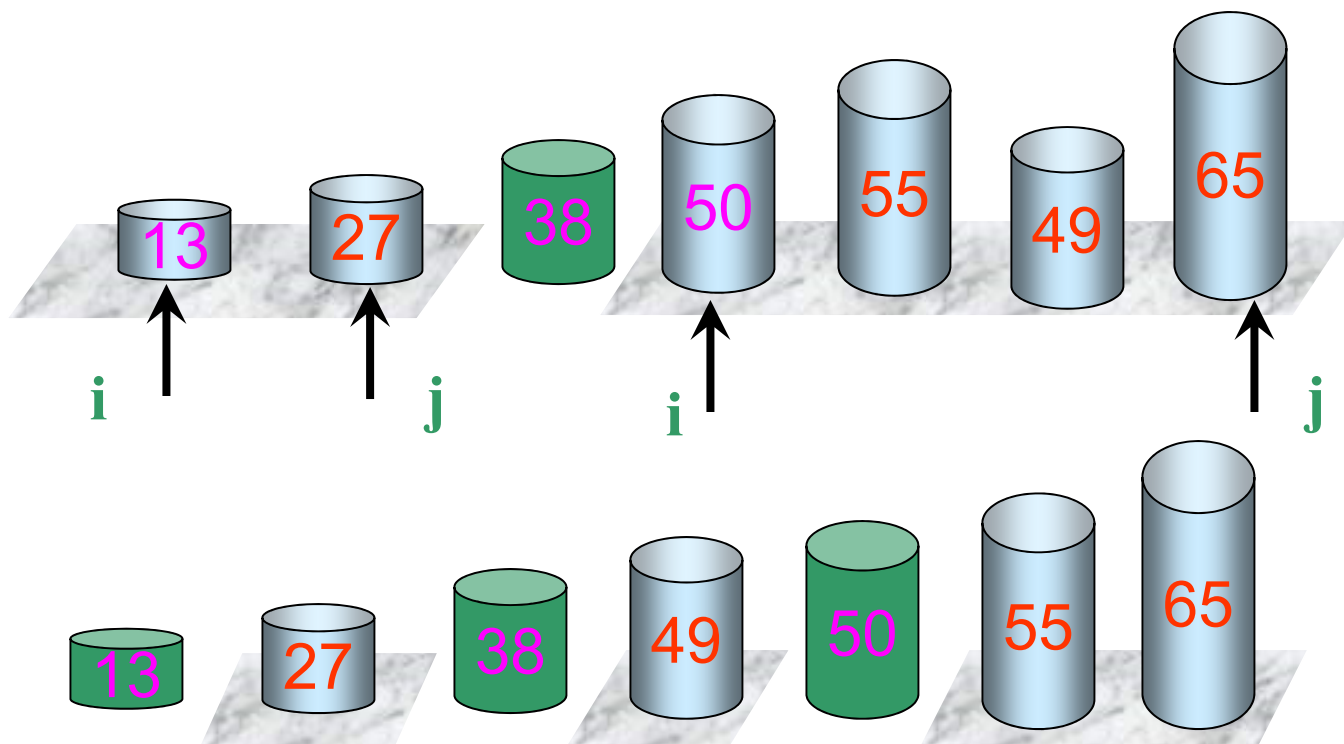
快速排序的分治策略是：

- **划分**：选定一个记录作为轴值，以轴值为基准将整个序列划分为两个子序列 $r_1 \dots r_{i-1}$ 和 $r_{i+1} \dots r_n$ ，前一个子序列中记录的值均小于或等于轴值，后一个子序列中记录的值均大于或等于轴值；
- **求解子问题**：分别对划分后的每一个子序列递归处理；
- **合并**：由于对子序列 $r_1 \dots r_{i-1}$ 和 $r_{i+1} \dots r_n$ 的排序是就地进行的，所以合并不需要执行任何操作。

一次划分示例



以轴值为基准将待排序序列划分为两个子序列后，
对每一个子序列分别递归进行排序。



在**最好情况**下，每次划分对一个记录定位后，该记录的左侧子序列与右侧子序列的**长度相同**。

在具有 n 个记录的序列中，一次划分需要对整个待划分序列扫描一遍，则所需时间为 $O(n)$ 。

设 $T(n)$ 是对 n 个记录的序列进行**排序的时间**，每次划分后，正好把待划分区间划分为长度相等的两个子序列，则有：

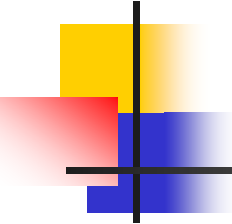
$$T(n) = O(n \log n)$$

在**最坏情况**下，待排序记录序列**正序或逆序**，每次划分只得到一个比上一次划分少一个记录子序列（另一个子序列为空）。

此时，必须经过 $n-1$ 次递归调用才能把所有记录定位，而且第 i 趟划分需要经过 $n-i$ 次关键码的比较才能找到第 i 个记录的基准位置，因此，总的比较次数为：

$$\sum_{i=1}^{n-1} (n-i) = \frac{1}{2}n(n-1) = O(n^2)$$

因此，时间复杂度为 $O(n^2)$ 。

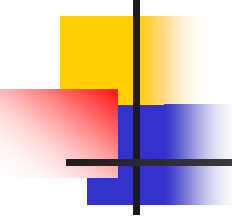


用Sherwood算法优化快速排序

- 基于Sherwood算法的优化
 - 每次随机选一个元素作为划分基准
 - 尽可能使划分基准不会是最大值或最小值

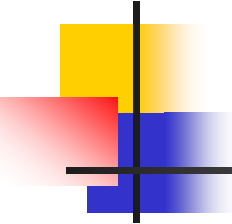
```
int RandomizedPartition(a[], p, r) {  
    int i = Random(p, r);  
    Swap(a[i], a[p]);  
    return Partition(a, p, r);  
}
```

优点是其计算时间复杂性对所有实例相对均匀。
但与其相对应的确定性算法比，平均复杂度没有改进。



随机算法四兄弟

1. 随机数值算法
2. Sherwood算法
3. **Las Vegas**算法
4. Monte Carlo算法



3. Las Vegas算法

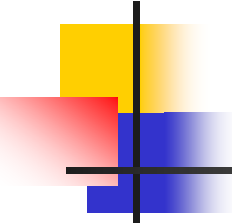
■ 思路:

- 随机算法运行一次得到正确解或者无解
- 可反复运行LV算法，直到得到正确解为止

```
void obstinate(Object x, Object y)
{   /* 反复调用Las Vegas算法LV(x,y),
    直到找到问题的一个解y */
    bool success= false;
    while (!success) success=LV(x, y);
}
```

↓

LV算法返回为bool型，有两参数：
1)输入； 2)成功时保存解。



3. Las Vegas算法

■ 性质：

- 一旦找到一个解, 该解一定是正确的
- 找到解的概率与算法执行时间成正比
- 增加对问题反复求解次数, 可使求解无效的概率任意小

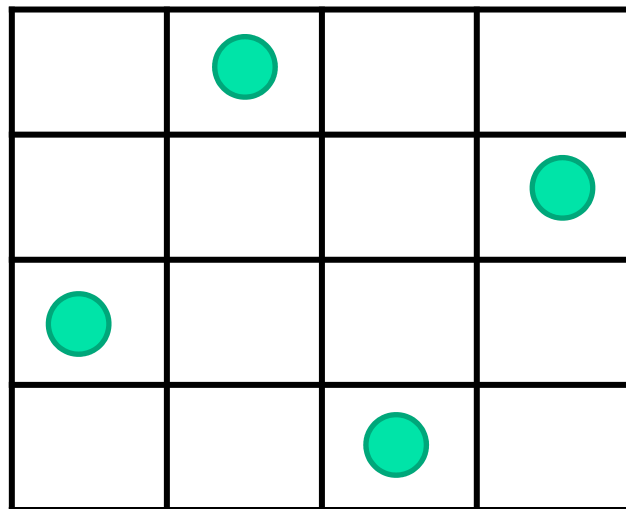
■ 应用范例：

- n 皇后问题
- 整数因子分解

用Las Vegas算法求解 n 皇后问题

■ n 皇后问题

- $n \times n$ 棋盘放 n 个皇后，使皇后之间相互不攻击
- 皇后位置无任何规律，不具有系统性，而更像是随机放置的
- 可采用随机算法求解



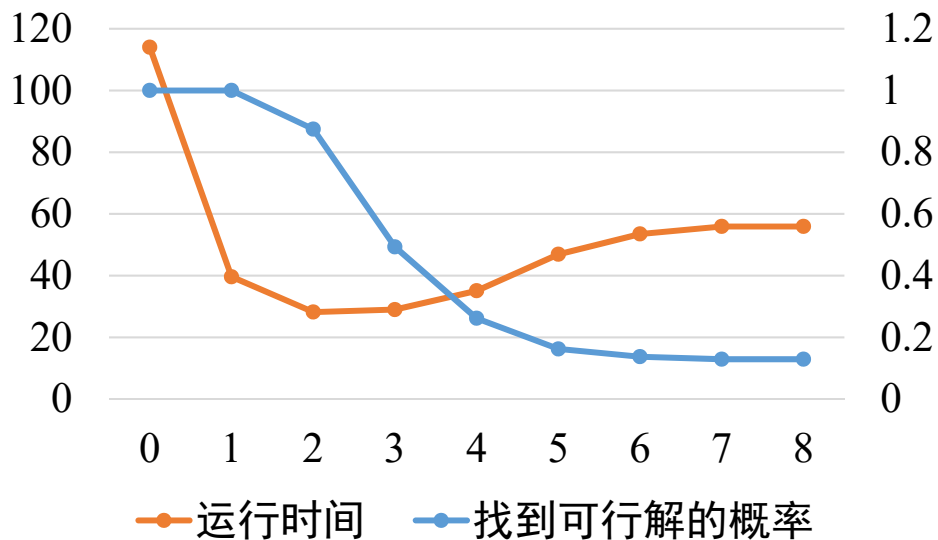
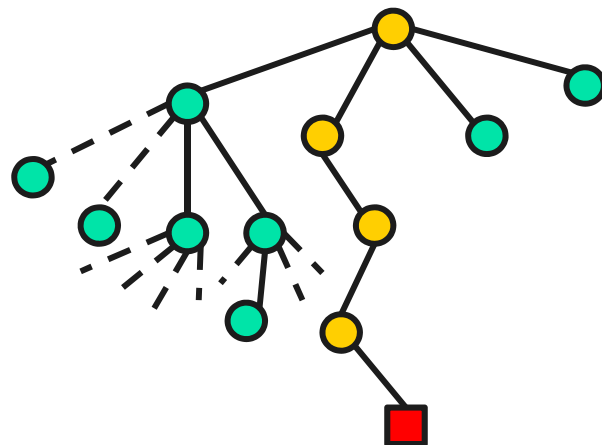
■ 基于Las Vegas算法的解法

- 随机地放置皇后
- 直至放完 n 个皇后(返回true)，或放不下皇后(返回false)

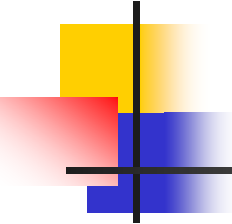
利用Las Vegas算法优化回溯法

■ 例子: n 皇后问题

- 解向量 $\langle x_1, x_2, \dots, x_n \rangle$
- 先用Las Vegas算法确定解向量的前 k 维
- 再用回溯法求解后 $n-k$ 维



- 利用Las Vegas算法+回溯法求解8皇后问题
- 当 $k=2$ 时性能最优

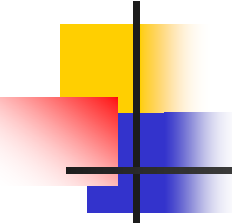


8皇后问题

■ 可得：

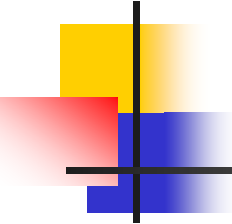
1. 随机放两个皇后，再回溯比完全用回溯**快**大约两倍；
2. 随机放三个皇后，再回溯比完全用回溯**快**大约一倍；
3. 随机放所有皇后，再回溯比完全用回溯**慢**大约一倍；

不能忽略产生随机数所需的时间，当随机放置所有的皇后时，八皇后问题的求解**大约有70%的时间**都用在了产生随机数上



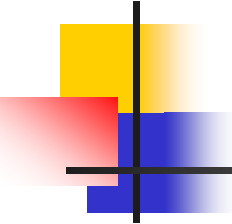
随机算法四兄弟

1. 随机数值算法
2. Sherwood算法
3. Las Vegas算法
4. **Monte Carlo算法**



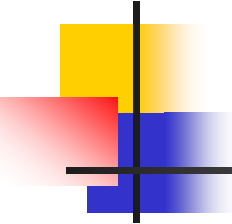
4. Monte Carlo算法

- 在实际应用中常会遇到一些问题，不论采用确定性算法或概率算法都**无法保证每次**都能得到正确的解答。
- 蒙特卡罗(MC)算法则在一般情况下可以保证对问题的所有实例都以**高概率给出正确解**，但是通常无法判定一个具体解是否正确。
- MC算法偶尔会出错，无论任何输入实例，总可以高概率找一正确解。即MC算法总是给出解，此解偶尔可能是不正确，也无法有效判定该解是否正确。



4. Monte Carlo算法

- 设 p 是一个实数，且 $1/2 < p < 1$ 。如果一个MC算法对于问题的任一实例得到正确解的概率不小于 p ，则称该MC算法是 p 正确的，且称 $p-1/2$ 是该算法的优势。
- 如果对于同一实例，蒙特卡罗算法不会给出2个不同的正确解答，则称该蒙特卡罗算法是一致的。
- 如果重复运行一个一致的 p 正确的MC算法，每次都进行随机选择，即可使产生不正确解的概率 $\rightarrow$ 任意小。
- 有些蒙特卡罗算法除了具有描述问题实例的输入参数外，还具有描述错误解可接受概率的参数。这类算法的计算时间复杂性通常由问题的实例规模以及错误解可接受概率的函数来描述。



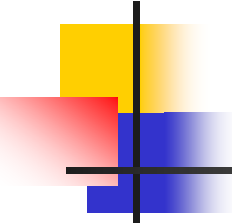
4. Monte Carlo算法

■ 性质：

- 主要用于求解需要准确解的问题
- 算法可能给出错误解
- 获得精确解概率与算法执行时间成正比

■ 应用范例

- 找主元素
- AMS物理实验
- 素数测试



主元素问题

■ 问题定义

- 若 n 元数组 T 中一半以上元素都是 x ，则 x 是 T 的主元素。
- 如何找出数组 T 的主元素

例如： $T[7]=\{3,2,3,2,3,3,5\}$ ，主元素为3

■ 传统解法

- 统计各元素出现的次数，时间复杂度 $O(n^2)$
- 是否还可以改进？